

Laboratorio

Procesamiento de imágenes

Visión por Ordenador I

Ingeniería Matemática e Inteligencia Artificial

Universidad Pontificia Comillas, ICAI



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

Laboratorio

Procesamiento de imágenes

Profesor:

Jaime Boal Martín-Larrauri
Lionel Güitta
Daniel Pinilla
Luis Arias
José María Bengochea
Mario Triviño

Email

jboal@comillas.edu
lglopez@comillas.edu
dpinilla@comillas.edu
learias@comillas.edu
jmbengochea@comillas.edu
mtrivino@comillas.edu

Cover:

Flying aboard Voyagers 1 and 2 are identical "golden" records, carrying the story of Earth far into deep space. This gold aluminum cover was designed to protect the gold-plated records from micrometeorite bombardment, but also serves a double purpose in providing the finder a key to playing the record. Image Credit: NASA/JPL-Caltech



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

Contents

1	Sesión 3: Procesamiento de imágenes	1
1.1	Materiales	1
1.2	Apartados de la práctica	1
1.3	Observaciones	1
1.4	Qué va a aprender	1
1.5	Evaluación	2
2	Apartado A: Filtro Gaussiano y Detección de bordes: Sobel y Canny	3
	Tarea A.1: Implementación de filtro Gausiano	3
	Tarea A.2: Aplicación de filtro Gausiano	4
	Tarea A.3: Implementación de filtro Sobel	4
	Tarea A.4: Aplicación de filtro Sobel	4
	Tarea B.5: Implementación de filtro Canny	5
	Tarea A.6: Aplicación de filtro Canny	5
	Preguntas	5
3	Apartado B: Operadores morfológicos	7
	Tarea B.1: Implementación de operadores morfológicos	7
	Preguntas	7

Sesión 3: Procesamiento de imágenes

1.1. Materiales

En esta práctica se trabajará con los siguientes recursos (puede encontrarlos en la sección de Moodle *Laboratorio/Sesión 3*):

- **partA-B_to-do.ipynb**: notebook con el código que deberá desarrollar en los apartados A y B.
- **helpers**: Archivos con métodos que le ayudarán a obtener los resultados.
 - `utils.py`
- **data**: carpeta con imágenes para trabajar durante la práctica.

1.2. Apartados de la práctica

La Sesión 3 del laboratorio está dividida en los siguientes apartados:

- **Librerías**: Importación de las librerías que se utilizan en la sesión. Se recomienda realizar la importación en una celda inicial para mantener la organización del Notebook.
- **Apartado A**: Filtro Gaussiano y Detección de bordes.
- **Apartado B**: Operadores Morfológicos.

1.3. Observaciones

Aunque el guion de la práctica y los comentarios en Markdown del Notebook estén escritos en español, observe que todo aquello que aparece en las celdas de código está escrito en inglés. Es una buena práctica que todo su código esté escrito en inglés.

Aquellas partes del código que deberá completar están marcadas con la etiqueta **TODO**.

Es muy importante que trabaje consultando la documentación de OpenCV¹ para familiarizarse de cara al examen. Tenga en cuenta que en los exámenes no podrá utilizar herramientas de ayuda como Copilot.

1.4. Qué va a aprender

Al finalizar esta práctica, implementará filtros Gaussianos, y aplicará métodos de detección de bordes como Sobel y Canny. Además, comparará sus implementaciones con las funciones predefinidas en `skimage`. También utilizará operadores morfológicos y comprenderá su utilidad.

¹Documentación de OpenCV: <https://docs.opencv.org/4.x/index.html>

1.5. Evaluación

La nota que obtenga en esta sesión de laboratorio será la misma que para ambos miembros de la pareja. Los apartados de la práctica serán evaluados como refleja la Tabla 1.1.

Tarea	Valor	Resultado
Pregunta A.1	2.5	
Pregunta A.2	3.0	
Pregunta A.3	2.5	
Pregunta B.1	2.0	
Total	10.0	

Table 1.1: Valoración de los apartados de la práctica.

Apartado A: Filtro Gaussiano y Detección de bordes: Sobel y Canny

En este apartado, trabajará con filtros para procesar imágenes. Los conceptos clave de este apartado son:

- **Filtro o Kernel:** es una matriz $n \times n$ que se desliza de forma ordenada por la imagen. Según sus valores, un filtro puede servir para difuminar una imagen, extraer sus bordes, etc.
- **Convolución.** Es una operación matemática que en visión por ordenador sirve para modificar una imagen. Toma los valores del kernel y los de la imagen en la posición (i, j) para producir el valor del píxel (i, j) de la nueva imagen.¹
- **Imagen:** El elemento sobre el que se aplica la operación.

Trabjará en la definición de tres filtros: Gaussiano, Sobel y Canny. Una vez definidos, deberá aplicar cada uno de los filtros sobre las imágenes de la carpeta data. A continuación se añaden algunos comentarios para resolver la tarea:

Tarea A.1: Implementación de filtro Gaussiano

Para aplicar el filtro Gaussiano, deberá tener cuenta los siguientes factores:

- **Tamaño del filtro.** Como podrá comprobar en los argumentos de la función, el tamaño del filtro puede (o no) venir dado. Cuando el tamaño del filtro no se pase como argumento, deberá obtener sus dimensiones a partir del valor de σ . Puede dar por supuesto que el efecto del filtro Gaussiano se considera extinto más allá de $\pm 4\sigma$. Además, tenga en cuenta que se busca un filtro $n \times n$ donde n es impar. La Figura 2.1 ilustra cómo aumenta el tamaño del filtro (*kernel size*, K) cuando aumenta σ .

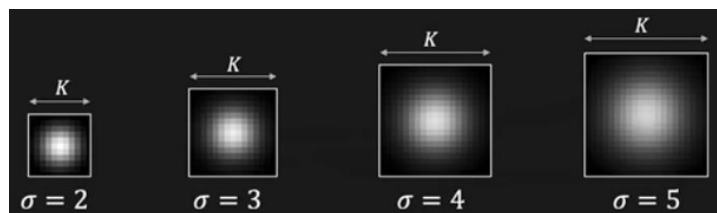


Figure 2.1: Tamaño del kernel en función de σ .

Con las restricciones y condiciones mencionadas, deberá definir el tamaño del filtro de tal forma que se cumpla lo indicado en la Ecuación 2.2:

¹Ejemplos animados de convoluciones: <https://gitlab.com/brohrer/convolution-2d-animation/-/tree/main>

$$\text{shape} = [L, L] \quad (2.1)$$

$$L = f(\sigma) \quad (2.2)$$

- **Coordenadas.** Con `np.mgrid()`, obtenga los valores de las coordenadas para x e y como dos arrays diferentes. Visite la documentación del método para visualizar un ejemplo de lo que se le pide.
- **Fórmula.** Defina la fórmula del filtro Gaussiano para cada elemento (i, j) siguiendo la Ecuación 2.3.

$$G(i, j) = \frac{1}{2\pi\sigma^2} e^{-\frac{i^2+j^2}{2\sigma^2}} \quad (2.3)$$

- **Convolución.** Aplique sobre el argumento `img` la operación de convolución con el filtro que acaba de definir. Utilice el método de OpenCV `cv2.Filter2D()`.

Tarea A.2: Aplicación de filtro Gaussiano

Aplique el filtro que acaba de definir al conjunto de imágenes de la carpeta `data`. Defina las variables necesarias para llamar al método. Puede utilizar una *list comprehension* para trabajar de forma eficiente. Como resultado de aplicar el filtro, debería obtener imágenes difuminadas o desenfocadas.

Tarea A.3: Implementación de filtro Sobel

Para aplicar el filtro Sobel, deberá tener cuenta los siguientes factores:

- **Imagen en escala de grises.** Trabaje con la imagen en escala de grises. Para ello, puede usar el método `cv2.cvtColor()` (que ya utilizó en la primera sesión de laboratorio).
- **Filtro Gaussiano.** Utilice el método que ha definido anteriormente. Sea cuidadoso proporcionando cada argumento para que sea heredable (es decir, utilice variables como argumentos).
- **Bordes verticales.** Obtenga los bordes verticales utilizando el filtro dado (`filter`). De nuevo, utilice el método `cv2.Filter2D()`.
- **Transformación del filtro.** Transforme el filtro para obtener los bordes en la dirección ortogonal.
- **Bordes horizontales.** Con el filtro transformado, utilice de nuevo `cv2.Filter2D()` para obtener los bordes ortogonales.
- **Composición de bordes.** Obtenga los bordes de la imagen como una composición de los bordes detectados en cada dirección. Puede utilizar para ello `np.hypot()`.
- **Orientación.** Obtenga la orientación de los bordes con `np.arctan2()`.

Tarea A.4: Aplicación de filtro Sobel

Aplique el filtro que acaba de definir al conjunto de imágenes de la carpeta `data`. Defina las variables necesarias para llamar al método. Puede utilizar una *list comprehension* para trabajar de forma eficiente. Como resultado de aplicar el filtro, debería obtener un resultado similar al de la Figura 2.2. El filtro que debe introducir como argumento de la función es:

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$



Figure 2.2: Ejemplo de bordes detectados usando el filtro Sobel.

Tarea A.5: Implementación de filtro Canny

El filtro Canny aporta un resultado más refinado para detectar bordes que el obtenido por Sobel. Para implementar el filtro Canny, deberá seguir los siguientes pasos:

- **Filtro Sobel.** Realice una llamada al método del filtro Sobel que ha definido en los pasos anteriores. De nuevo, sea cuidadoso proporcionando cada argumento para que sea heredable (es decir, utilice variables como argumentos).
- **Non Max Supresion.** Utilice el método `non_max_supression()` definido en el archivo `utils.py`. Inspeccione los argumentos que necesita utilizar este método para poder ejecutarlo de forma adecuada. Este método se encarga de refinar los bordes, devolviendo una imagen en escala de grises.

Tarea A.6: Aplicación de filtro Canny

Aplique el filtro que acaba de definir al conjunto de imágenes de la carpeta `data`. Defina las variables necesarias para llamar al método. Puede utilizar una *list comprehension* para trabajar de forma eficiente. Como resultado de aplicar el filtro, debería obtener los bordes de las imágenes de forma similar a cuando aplicó el filtro Sobel.

Preguntas

Pregunta A.1

Añada ruido a las imágenes de la carpeta `data`. Compare los resultados que obtiene al aplicar su filtro Sobel **con y sin filtro Gausiano**. Para añadir ruido, puede sumar a la imagen original una matriz de las dimensiones adecuadas. Puede utilizar `np.random.normal()`, pero tenga en cuenta que este proceso puede hacer que algunos valores de la imagen salgan del rango `[0, 255]` (tome las medidas necesarias para corregir esto).

Pregunta A.2

Utilice la librería `scikit-image` y compare el efecto de los filtros Sobel, Canny y Prewitt sobre las imágenes de la carpeta `data`. ¿Qué diferencias observa entre los filtros? ¿Puede obtener alguna conclusión y/o patrón?

Pregunta A.3

Añada ruido impulsivo (“sal y pimienta”) a una imagen de la carpeta data, corrompiendo un 5% de sus píxeles (fije su valor a 0 o a 255 de forma aleatoria). Aplique sobre la imagen ruidosa un filtro Gausiano (`cv2.GaussianBlur()`) y un filtro de mediana (`cv2.medianBlur()`), ambos con el mismo tamaño de kernel. Compare los resultados con la imagen original y explique por qué se comportan de forma distinta. ¿Qué ocurriría si aumentase el tamaño del kernel o el porcentaje de píxeles corrompidos?

Apartado B: **Operadores morfológicos**

Tarea B.1: Implementación de operadores morfológicos

En esta sección debe implementar los operadores morfológicos que se indican a continuación.

Preguntas

Pregunta B.1

Implemente los operadores morfológicos dilatación y erosión. Utilice para ello un kernel de 3x3 con valores 255 (blanco) como structuring element. No olvide binarizar su imagen antes de aplicar los operadores morfológicos.